Prof. Ashwani Mishra
Assistant Professor
CSE Department

1. Introduction to Angular

What is Angular?

Angular is a TypeScript-based front-end framework developed by Google used to build dynamic Single Page Applications (SPA).

In Angular:

- UI updates without refreshing the page
- Applications are component-based
- Used in MEAN stack frontend

Real Life Example

Example: Gmail

When you open an email:

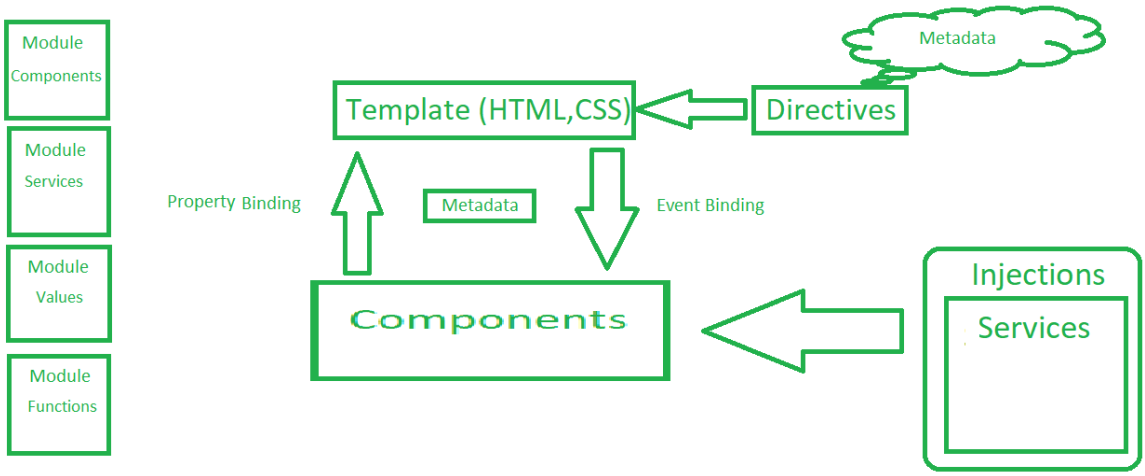
- Page does not reload
- Only data changes dynamically

This is Single Page Application behavior, similar to Angular apps.

Features of Angular

Feature	Description
Component Based	UI divided into reusable components
TypeScript	Strongly typed JavaScript
Data Binding	Connects UI and logic
Dependency Injection	Manages services
Routing	Navigation between pages

Angular Architecture



Modules

↓

Components

↓

Templates

↓

Services

2. Setting up Angular Application

Step 1 Install Node.js

Download:

https://nodejs.org

Check version:

node -v
npm -v

Step 2 Install Angular CLI

Angular CLI helps create and manage Angular projects.

npm install -g @angular/cli

Check installation:

ng version

Step 3 Create Angular Project

ng new student-app

Select:

Routing: Yes
Stylesheet: CSS

Step 4 Run Angular Project

cd student-app
ng serve

Open browser:

http://localhost:4200

Angular Project Structure

```
student-app
 ├── src
 |   ├── app
 |   |   ├── app.component.ts
 |   |   ├── app.component.html
 |   |   ├── app.module.ts
 |   ├── index.html
 |   ├── main.ts
```

3. Components, Modules, and Services

Components

A **Component** controls a **part of the user interface**.

Example:

- Login Component
- Dashboard Component
- Student Component

Generate Component:

ng generate component student

Files created:

student.component.ts
student.component.html
student.component.css
student.component.spec.t

Component Example

```
import { Component } from '@angular/core';

@Component({

 selector: 'app-student',

 template: '<h2>Student Component</h2>'

})

export class StudentComponent {}
```

Output:

Student Component

Services

Services provide **business logic and data sharing**.

Example uses:

- API calls
- Data sharing between components

Create Service

```
ng generate service student
```

Service Example

```
import { Injectable } from '@angular/core';
@Injectable({
 providedIn: 'root'
})
export class StudentService {
 getStudents(){
   return ["Rahul","Amit","Riya"];
 }
}
```

4. Data Binding and Templates

What is Data Binding?

Data binding connects **component data with HTML template**.

Types of Data Binding

Type	Syntax
Interpolation	{{ }}
Property Binding	[property]
Event Binding	(event)
Two Way Binding	[(ngModel)]

Interpolation Example

```
<h1>{{title}}</h1>
```

Component

```
title = "Angular Tutorial";
```

Output:

Angular Tutorial

Property Binding Example

```
<img [src]="imageURL">
```

Event Binding Example

```
<button (click)="showMessage()">Click</button>
```

Component

```
showMessage(){
 alert("Button Clicked");
}
```

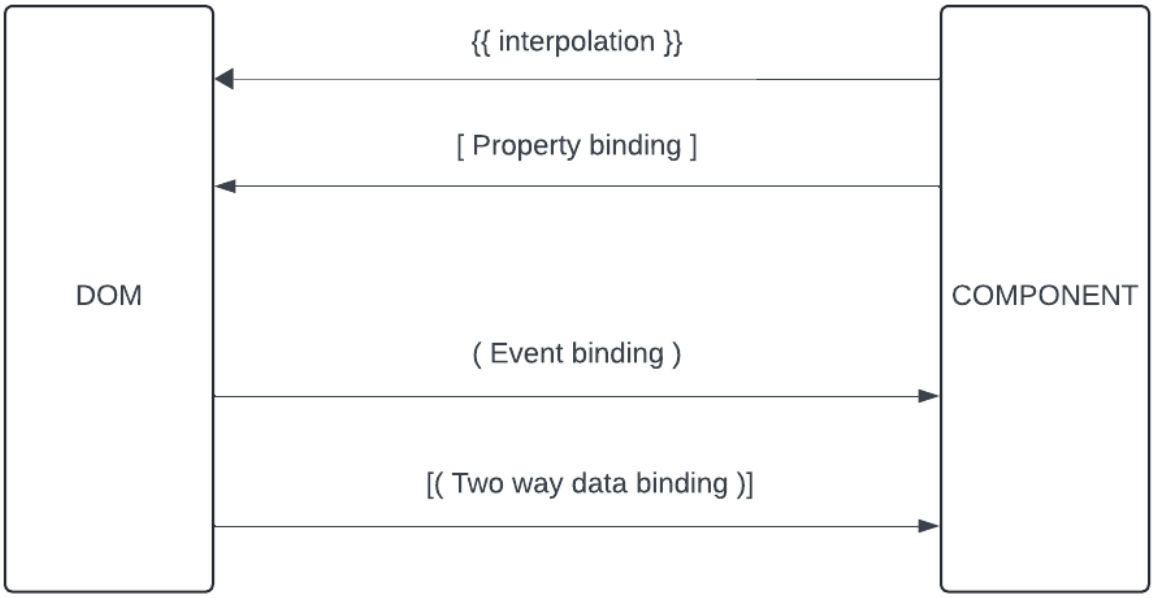
Component

```
showMessage(){
 alert("Button Clicked");
}
```

Two Way Data Binding

```
<input [(ngModel)]="name">
<p>{{name}}</p>
```

Data Binding Diagram



Data Binding In Angular

5. Forms and Validation

Angular supports:

1 Template Driven Forms
2 Reactive Forms

Example Login Form

```
<form #loginForm="ngForm">

<input name="username" ngModel required>

<input type="password" name="password" ngModel required>

<button>Login</button>

</form>
```

Validation Example

```
<input name="email" ngModel required email>

<div *ngIf="email.invalid">

Invalid Email

</div>
```

Form Validation Types

Validation	Example
Example	required
Email	email
Min Length	minlength
Pattern	pattern

6. Routing and Navigation

Routing allows navigation between pages.

Example pages:

/home

/about

/contact

Create Routing

ng generate component home

ng generate component about

Router Configuration

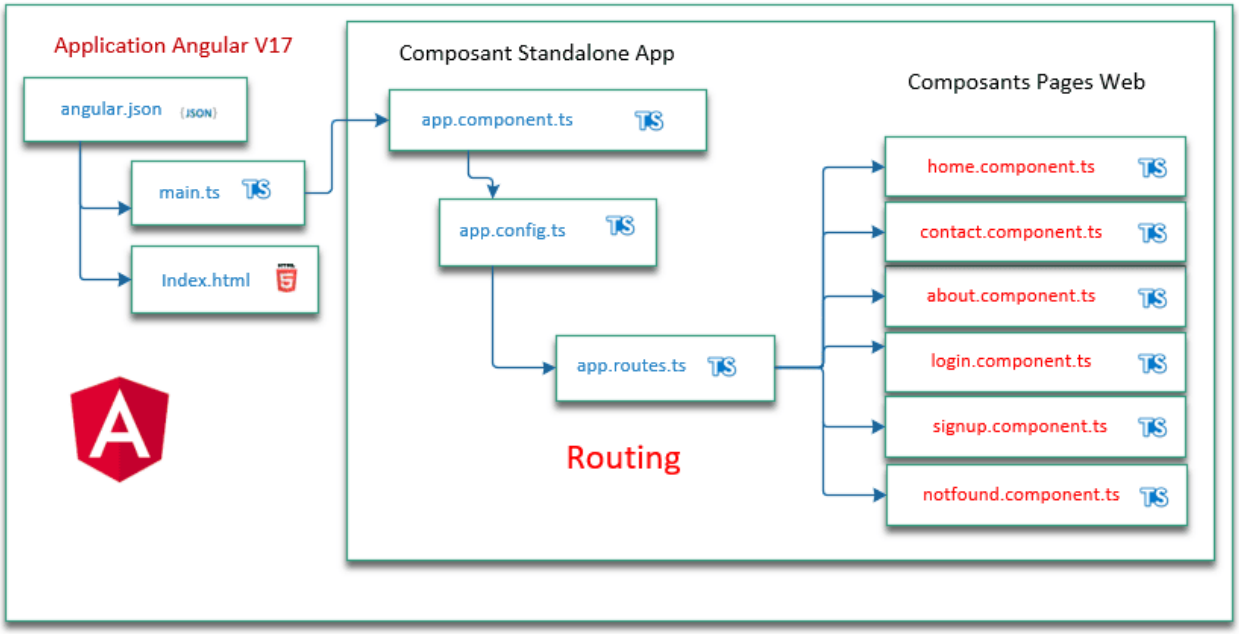
```
import { NgModule } from '@angular/core';

import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
```

```
 { path:'home', component:HomeComponent },
 { path:'about', component:AboutComponent }
];
@NgModule({
 imports:[RouterModule.forRoot(routes)],
 exports:[RouterModule]
})
export class AppRoutingModule {}
```

Navigation Links:

```
<a routerLink="/home">Home</a>
<a routerLink="/about">About</a>
<router-outlet></router-outlet>
```

Angular Routing Flow



7. HTTP and Observables

Angular communicates with backend using HTTP Client.

Example backend:

Node.js + Express API

Import HTTP Module

```
import { HttpClientModule } from '@angular/common/http';
```

Add to module:

```
imports:[HttpClientModule]
```

Example API Call

```
import { HttpClient } from '@angular/common/http';

constructor(private http:HttpClient){}

getStudents(){
 return this.http.get("http://localhost:3000/students");
}
```

Observables

Observables handle **asynchronous data streams**.

Example:

```
this.service.getStudents().subscribe(data=>{
 console.log(data);
});
```

Flow:

Request → Server → Response → Observable → Subscribe

8. Building Complete Frontend for MEAN Stack

Example Project:

Student Management System

Frontend using Angular.

Features

Feature	Description
Add Student	Form
View Students	Table
Edit Student	Update form
Delete Student	Remove record

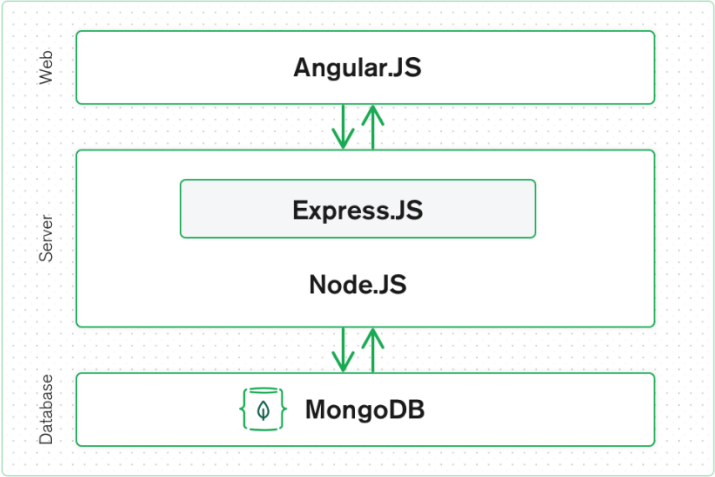
Example Student List UI

```
<h2>Student List</h2>
<ul>
<li *ngFor="let student of students">
 {{student.name}}
</li>
</ul>
```

Component

```
students = [
 {name:"Rahul"},
 {name:"Amit"},
 {name:"Riya"}
];
```

Angular + Node + MongoDB Flow



Flow:

Angular (Frontend)

↓

HTTP Request

↓

Node.js + Express API

↓

MongoDB Database

↓

Response → Angular UI